Branch • Optimal Keypoint Annotation for Beacon Pose Extraction

https://gemini.google.com/app/d57aa436effec16d

User prompt: In doing pose extraction for a beacon, one of our contributors suggested using the black diamonds instead of the body edges. Should keypoints be laid on the corners of each of the 4 diamonds, 16 total keypoints, or just in the center of each diamond, 4 total keypoints?

Response: You should place keypoints on the **corners of each diamond (16 total keypoints)** rather than just their centers.

Here is why 16 corner keypoints are significantly better for pose extraction (PnP / Perspective-n-Point):

- **Single-Face Pose Constraints:** From any given camera angle, typically only **1 or 2 faces** of the beacon are visible (as seen in your image).
    - If you use only centers, a view seeing just 1 face provides **1 point** (underconstrained; PnP requires a minimum of 4 non-collinear coplanar/non-coplanar points). Even seeing 2 faces provides only **2 points**, which is still insufficient to solve a 6-DoF pose.
    - If you use diamond corners, seeing just **one face** provides 4 non-collinear coplanar 2D-to-3D correspondences—enough for standard planar PnP / homography decomposition. Seeing two faces gives up to 8 points across two distinct planes, yielding a highly accurate, fully-constrained 3D pose.
- **Perspective Invariance:** Under perspective projection, the visual center (centroid) of a planar polygon in an image does not project to the true 3D geometric center. Corner vertices, however, are projective invariants (a physical corner always maps directly to its 2D projection).
- **Localization Accuracy:** Corner junctions provide sharp gradients in two orthogonal directions, making them far easier and more consistent for a CNN or subpixel corner detector to localize precisely compared to estimating the interior center of a uniform black patch.

Recommended Annotation Strategy

- Define the 3D model coordinates for all 16 corners relative to the beacon frame.
- Label the 4 corners of each visible diamond consistently (e.g., top, right, bottom, left relative to that specific face).
- When running PnP, use RANSAC-PnP (`cv2.solvePnPRansac`) with the subset of visible diamond corners.

User prompt: Given the beacon is symetric on X & Y axis and has these diamonds on all 4 sides. How should I go about configuring its KPs in RoboFlow?

Response: Because the beacon is physically 4-fold rotationally symmetric (all 4 sides look identical), a keypoint detector cannot tell which side is "North", "East", "South", or "West". If you create 16 distinct global keypoints (e.g., `face1_top`, `face2_top`), your model will fail because an annotator cannot consistently tell them apart.

To handle this cleanly in Roboflow and downstream PnP, choose one of two strategies:

Strategy A: Generic View-Relative Faces (Recommended)

Instead of assigning cardinal directions, define keypoints relative to the **visible faces** in the bounding box. Since at most 2 faces are clearly visible at once:

1. **Create 8 Keypoints (2 Faces $\times$ 4 Corners):**
    - Face 1 (e.g., Primary / Left-facing):

        - `f1_top`, `f1_bottom`, `f1_left`, `f1_right`
    - Face 2 (e.g., Secondary / Right-facing):
        - `f2_top`, `f2_bottom`, `f2_left`, `f2_right`
2. **Setup in the Roboflow Template:**
    - Click on the canvas to place a diamond shape (4 points) for Face 1, and connect them with edges so it forms a diamond.
    - Place another 4 points next to it for Face 2 and connect them.
3. **Annotation Rule:**
    - If only **1 face** is visible: Annotate `f1_*` points; mark `f2_*` points as occluded/absent.
    - If **2 faces** are visible: Annotate the left-most diamond as `f1_*` and the right-most diamond as `f2_*`.
4. **PnP Ambiguity Resolution:**
    - Because of 4-fold symmetry, the position (X, Y, Z) is unique, but the yaw rotation has an inherent 4-fold ambiguity (90° increments). For beacon docking/guidance, you typically only care about relative position (X, Y, Z) and tilt/orientation relative to the face normal.

Strategy B: Single-Diamond Detector (Alternative)

If you don't want to define multiple faces inside a single beacon bounding box:

1. **Object Class:** Change the detected object class from `beacon` to `beacon_marker` or `diamond`.
2. **4 Keypoints per Object:**
    - `top_corner`
    - `bottom_corner`
    - `left_corner`
    - `right_corner`
3. **Setup in Roboflow:**
    - Place 4 points in a diamond pattern and connect them with edges.
4. **Inference / PnP Workflow:**
    - The model detects every visible diamond separately.
    - If 1 diamond is detected: You immediately have 4 points coplanar in 3D.
    - If 2 diamonds are detected: You group them by beacon proximity, giving 8 points across two known angled planes.

Roboflow "Symmetries" Tab Configuration

In the modal shown in your screenshot, click the **Symmetries** tab:

- Pair the **left** and **right** corners of each diamond (e.g., `f1_left` $\leftrightarrow$ `f1_right`) as horizontal reflection symmetries.
- This ensures that when Roboflow applies horizontal flip augmentations during training, the labels automatically mirror without corrupting left/right orientation.

User prompt: When doing YOLO Pose can it do just BBOX detect for some classes and actual pose for others?

Response: **No, not out-of-the-box in standard YOLO Pose.**

In YOLO Pose architectures (such as Ultralytics YOLOv8/YOLO11 Pose), the model architecture uses a fixed keypoint regression head defined by a single dataset-wide parameter `kpt_shape: [N, 2]` or `[N, 3]`. Because that output tensor size is fixed per bounding box, **every detection candidate across all classes is forced to output the exact same number of keypoints (N).**

However, you can achieve this behavior depending on which workaround fits your pipeline:

Option 1: Visibility Zero-Padding (Single Model Workaround)

If you must use a single YOLO-Pose model for both pose objects (e.g., `beacon`) and bounding-box-only objects (e.g., `person`, `obstacle`):

- Set the global `kpt_shape` to match your pose object's keypoint count (e.g., $N = 4$ or 8).
- In your label text files, for the bbox-only classes, set all keypoint coordinates and visibility flags to zero:

```
# Class 0 (Beacon with 4 keypoints): class x y w h kp1_x kp1_y 2 kp2_x kp2_y 2 ...
0 0.5 0.5 0.2 0.2  0.48 0.45 2  0.52 0.45 2  0.52 0.55 2  0.48 0.55 2

# Class 1 (Bbox-only object): class x y w h (all keypoints set to 0 0 0)
1 0.2 0.3 0.1 0.1  0 0 0  0 0 0  0 0 0  0 0 0
```

- Because YOLO only calculates keypoint loss on annotations where keypoint visibility $v > 0$, the network ignores keypoint gradients for Class 1.
- During inference, simply ignore the `results.keypoints` field whenever `class == 1`.

Option 2: Two-Stage Pipeline (Recommended for Production)

If your bbox-only objects are distinct from your pose targets (e.g., detecting obstacles, landing pads, or drones vs. fine-grained beacon pose):

1. **Model A (Standard YOLO Detect):** Fast, lightweight model detecting all primary object classes.
2. **Model B (YOLO Pose):** Dedicated only to the beacon/pose class, run either on crops or as a specialized detector.

This avoids wasting model capacity regressing dummy keypoints for objects that don't need them and prevents annotation headaches in tools like Roboflow (which expect a complete skeleton layout per project class).

User prompt: Can RoboFlow train a YOLO pose model?

Response: Yes, directly in Roboflow's UI.

As shown right on your screen:

- The **YOLO26** option on the right is Ultralytics' YOLO architecture built directly into Roboflow's auto-train workflow.
- Because your project type is set to **Keypoints / Pose**, selecting **YOLO26** will configure and train a YOLO pose estimation model directly on Roboflow's cloud GPUs.
- Alternatively, **Roboflow RF-DETR** (on the left) is Roboflow's transformer-based keypoint model, which is typically higher accuracy for keypoints but heavier to infer.

Two Requirements Before You Can Click "Train"

Looking at the bottom of your screen, there are two issues to resolve before launching:

1. Fix the Dataset Split ("Split is below the minimum"):
    - Your current dataset is set to **100% / 0% / 0%** (Train / Valid / Test).
    - Click **"Adjust splits"** and set it to a standard ratio (e.g., **70% Train / 20% Valid / 10% Test** or **80% / 20% / 0%**). Roboflow requires a validation set to evaluate training loss and mAP.
2. **Generate a Version:**
    - Once the split is saved, Roboflow generates Version 1 (v1), which unlocks the training button.

Alternative: Exporting and Training Locally / in Colab

If you ever run out of Roboflow training credits or want to customize hyper-parameters (such as custom loss weightings for keypoint OKS, or running YOLOv8/YOLO11 locally):

1. Go to **Versions** → **Export Dataset**.
2. Choose **YOLOv8 Pose** format.
3. Train locally with Ultralytics using a single line:

```
from ultralytics import YOLO

model = YOLO("yolo11n-pose.pt")  # or yolov8n-pose.pt
model.train(data="data.yaml", epochs=100, imgsz=640)
```

User prompt: YOLOv11 nano vs YOLO26 nano Qualcomm GPU OpenCL perf for pose?

Response: **YOLO26 nano is noticeably faster and better suited for Qualcomm Adreno OpenCL acceleration than YOLO11 nano for pose estimation.**

The key architectural differences and their real-world impact on Qualcomm runtimes (e.g., Tengine/MNN OpenCL, QNN GPU backend, or ONNX Runtime OpenCL) break down as follows:

1. Removal of Distribution Focal Loss (DFL)

- **YOLO11 nano:** Uses DFL (Distribution Focal Loss) in its detection/anchor head. In OpenCL, DFL introduces custom reshaping, slice operations, and `Softmax` over coordinate bins across thousands of anchor locations. These operations are memory-bandwidth-bound and cause inefficient kernel launches and texture stalls on Adreno mobile GPUs.
- **YOLO26 nano:** Completely eliminates DFL in favor of direct continuous coordinate regression. In OpenCL, this flattens the output head into standard fused linear/convolutional layers, dramatically reducing dispatch overhead and latency on Adreno shader cores.

2. End-to-End, NMS-Free Execution

- **YOLO11 nano:** Requires Non-Maximum Suppression (NMS). On Qualcomm platforms, NMS rarely runs cleanly on OpenCL because of sequential suppression and dynamic shape handling. The raw box + keypoint tensors must either:
    1. Synchronize back to the CPU (incurring a costly GPU-to-CPU DMA copy and CPU NMS stall).
    2. Fall back to custom OpenCL top-k kernels that don't scale well.
- **YOLO26 nano:** Native one-to-one Hungarian-style matching during training produces an **NMS-free** head. The model directly outputs the top candidate boxes and keypoint sets. The entire network executes end-to-end inside the GPU context without CPU synchronization barriers.

3. OpenCL Graph Fusion & Layer Compatibility

- **YOLO11 nano:** Features C3k2 and attention-style spatial blocks (C2PSA), which introduce multi-branch splits, transpositions, and concatenation layers. On Adreno OpenCL runtimes, branches that cannot be folded into standard Winograd/direct 2D convolution kernels trigger intermediate texture allocations and extra memory round-trips.
- **YOLO26 nano:** Streamlines feature aggregation into hardware-friendly, edge-first sequential blocks that fuse cleanly into 16-bit float (`cl_khr_fp16`) direct convolution kernels in OpenCL.

Expected Performance & Latency Profile (Snapdragon / Adreno)

Metric / Characteristic	YOLO11 nano (`yolo11n-pose`)	YOLO26 nano (`yolo26n-pose`)	Advantage
Adreno OpenCL Latency (640×640, FP16)	~9 – 14 ms	~5.5 – 8 ms	**~30–40% faster on YOLO26**
Pipeline Latency Bottleneck	CPU sync for NMS & keypoint association	Pure GPU forward pass	YOLO26
Export/Parser Stability (ONNX / QNN)	Can fail or require custom NMS graph surgery	Clean single-output static graph	YOLO26
Beacon Keypoint Localization (mAP / OKS)	High	Comparable / slightly superior	Tie / YOLO26

Recommendation for Beacon Pose

- **Choose YOLO26 nano** if you are deploying to a Qualcomm board (such as RB3 Gen 2, RB5, or Snapdragon flight controllers) via OpenCL or Qualcomm AI Hub / QNN. The removal of NMS and DFL eliminates the exact operations that traditionally degrade performance on Adreno GPUs.
- If deploying via OpenCL, ensure you enable **FP16 precision** (`CL_HALF_FLOAT` / `--fp16`), which unlocks the full arithmetic throughput of Adreno ALUs and halves memory bus saturation.

User prompt: How is nano given we're only dealing with a single, high contrast object with well defined corners?

Response: **Nano is the ideal model tier for this exact problem.**

Using larger variants (Small, Medium, or X-Large) for a single, high-contrast beacon with sharp diamond corners is unnecessary and will add latency and memory overhead without measurable accuracy gains.

Why Nano Excels Here

- **Low Visual Complexity:** Higher model tiers exist to untangle heavy occlusion, multi-scale clutter (like 80 COCO classes in busy crowds), and ambiguous feature boundaries. A stark black diamond set against a white beacon body on grass provides very sharp corner gradients that early convolutional layers detect effortlessly.
- **Pixel-Level Localization vs. Semantic Capacity:** Keypoint regression accuracy on crisp geometric corners depends primarily on **spatial resolution (input image size)**, not network depth or parameter count. An early layer in a 3-million parameter Nano model has all the edge-filtering capacity needed to locate diamond corners.
- **Better Training on Small Datasets:** With a single object and limited training images (e.g., your ~90-image dataset), larger models like Medium or X-Large will quickly overfit to specific background patches (e.g., specific grass textures or lighting). Nano acts as a regularizer, forcing the network to learn only the core geometric structure.

Where to Put Your Engineering Effort Instead

Rather than sizing up the model, optimize these two factors for millimeter-accurate PnP:

1. **Resolution over Model Depth:**
    - Run Nano at 640×640 (or even 800×800 if range is long). Giving Nano more pixels produces much tighter corner coordinates than giving a larger model a smaller 320×320 image.
2. **Subpixel Corner Refinement (Optional Post-Processing):**
    - Let Nano give you the coarse detection box and estimated corner points.
    - Because the diamonds are high-contrast black on white, pass Nano's predicted corner points through OpenCV's `cv2.cornerSubPix()` or a local gradient Harris step within a small window (5×5 px) around each keypoint before calling `cv2.solvePnP()`. This gives sub-pixel precision with zero neural network overhead.

User prompt: YOLOv11n seems to perform better off initial metrics.

Response: The comparison table highlights a substantial difference between the two models:

- **YOLOv11n-pose-t1: 93.8% mAP@50**, **95.2% Precision**, **90.9% Recall**, **93.0% F1**
- **YOLO26n-pose-t2: 84.8% mAP@50**, **93.8% Precision**, **68.2% Recall**, **78.9% F1**

The root cause of YOLO26n's lower score is **Recall (68.2% vs. 90.9%)**. While precision is almost identical (~94–95%), YOLO26n is missing or failing to detect ~23% more instances under the default evaluation threshold.

The disparity stems from three primary factors:

1. End-to-End One-to-One Matching Requires More Epochs

YOLO26 replaces traditional NMS with bipartite Hungarian-style one-to-one matching during training.

- Traditional anchor/NMS architectures like YOLOv11 use many-to-one label assignment (multiple anchors predict the same ground-truth box during training, then NMS filters them). This trains very quickly and converges to high recall in few epochs on small datasets (~90 images).
- NMS-free models require the network to learn both localization *and* mutual prediction suppression simultaneously. With default quick-training schedules, NMS-free models almost always trail in early-epoch recall before catching up with longer training.

2. Default Confidence Threshold Sensitivity

Because YOLO26 eliminates NMS, the network suppresses duplicate predictions by driving candidate confidence scores down. At standard validation cutoffs (e.g., confidence ≥ 0.25 or 0.5), valid detections with lower raw confidence scores get clipped, causing recall to drop sharply to 68.2%. If you lower the inference confidence threshold (e.g., to 0.10–0.15), YOLO26's recall often climbs to parity without flooding the output with duplicates.

3. Training Recipe Differences

Notice the line under the model names:

- YOLOv11 was trained with `Training Preprocessing · Aut...`
- YOLO26 was trained with `Training Preprocessing · Res...`

Differences in augmentations (e.g., auto-orient vs. resize/crop/mosaic) can significantly shift validation metrics on small datasets.

What to Do Next

1. **Test on Real Test Images:** Click into **View Model** for both and evaluate visual predictions on difficult validation frames. Check whether YOLO26n is truly failing to see the beacon or if it is merely outputting confidence scores just below Roboflow's default metric cutoff.
2. **If Deploying to Qualcomm OpenCL:**
    - If you need the cleanest OpenCL export and fastest on-device Adreno latency, YOLO26 is still architecturally superior. You can compensate for the recall gap by lowering your confidence threshold or training for additional epochs.
    - If deployment ease and raw initial accuracy right now matter most, **YOLOv11n** is clearly the superior checkpoint out-of-the-box on your current dataset split.
